

Theory

1. What is a balanced binary tree?

A balanced binary tree is a binary tree in which, for **every node**, the height difference between its left and right subtrees is smaller or equal to 1.

2. Given the representation of a binary tree: [null, 25, 40, 60, 80, 90, 120]

What is the left and right child of 60 (index 3)?

Ans: We know that

- If Node index is i ,
Left Child at $2i$, Right Child at $2i+1$

So, if Node is 3. Left Child is $2*3=6$ th node. So left child is 120
Right child is at $2*3+1$. So it's null.

3. You are given two sequences:

Inorder Traversal: [2, 5, 7, 10, 12, 15, 20]

Preorder Traversal: [10, 5, 2, 7, 15, 12, 20]

Construct the tree from the two given sequences.

Ans:

Preorder → first element is always the root

Inorder → left of root = left subtree, right of root = right subtree [Balancing]

```

  10
 /  \
5    15
 / \  / \
2  7 12 20
```

4. What is the maximum number of nodes in a binary tree of height H?

$$\text{Max node} = 2^{h+1} - 1$$

5. If a Binary Tree has 31 nodes, what is the minimum possible height (root at height 0)?

$$\text{Max node} = 2^{h+1} - 1$$

Since 31 has the minimum height, it will be smaller or equal to max node.

$$2^{h+1} - 1 \geq 31$$

$$2^{h+1} \geq 32$$

$$h+1 \geq 5$$

$$h \geq 4$$

Since it asks for minimum height $h+4$

6. If there are a total of 9 nodes in a tree, then what will be the total edges in that tree?

$$\text{Edges} = (\text{Nodes} - 1) = 8 \text{ edges}$$

7. If we mirror the above tree, what will be the new post-order traversal?



Ans: 12, 15, 7, 3, 5, 10

8. What is the degree of the above original tree?

2(The degree of a tree is the maximum number of children any node has)

9. Forming tree from traversal

a. Preorder + Inorder

Preorder: 1, 2, 4, 5, 3

Inorder: 4, 2, 5, 1, 3

Ans: Take first element of preorder → root

Find it in inorder → split into left/right subtree

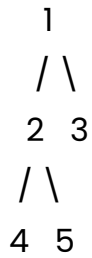
```

  1
 / \
2   3
 / \
4   5
  
```

b. Postorder + Inorder

Postorder: 4, 5, 2, 3, 1

Inorder: 4, 2, 5, 1, 3

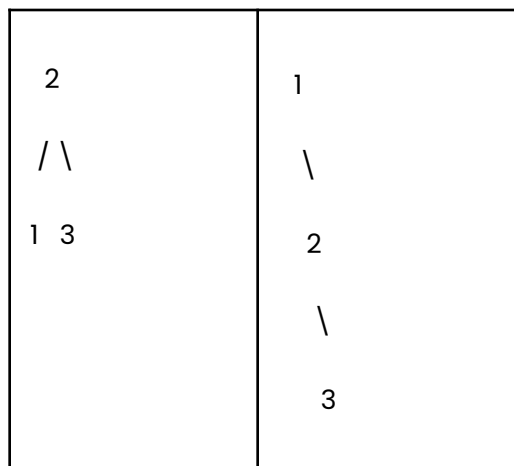


10. If we have two different Binary Trees with the same in-order traversal, which of the following is correct?

- A. They must be identical trees
- B. They must have the same height
- C. They must have the same preorder traversal
- D. They can have different structures

Answer: D

Example:



Both have the same inorder traversals but their structure is different. One is balanced, another isn't

11. Maximum number of nodes in level is- 2^i

12. Find the correct statements:

- A.** In a Perfect Binary Tree of height h , the total number of leaf nodes is 2^h .
- B.** In a Complete Binary Tree, if a node doesn't have a right child, it must not have a left child as well.
- C.** In a Binary Tree, consider two sibling nodes: A, B. The level of A = 3, and the level of B = 2.
- D.** All Binary Search Trees are Binary Trees, but not all Binary Trees are Binary Search Trees.

Ans: D

- A : Total leaf nodes = 2^h only if height is counted from 0; otherwise formula may vary.
Total number of nodes = $2^{h+1} - 1$;
- B : In a complete BT, a node can have a left child without a right child.
- C : Siblings are always at the same level.
- D ☒: Definitionally true.

13. Type of trees

Full / Proper / Strict Binary Tree • Every node has 0 or 2 children • No node has exactly one child	<pre> 1 / \ 2 3 / \</pre>
--	--------------------------------

	4 5
Complete Binary Tree • All levels are full except possibly the last • Last level is filled from left to right	<pre> 1 /\ 2 3 /\ 4 5 </pre>
Perfect Binary Tree • All internal nodes have 2 children • All leaf nodes are at the same level • Total nodes = $2^{(h+1)} - 1$	<pre> 1 /\ 2 3 /\ /\ 4 5 6 7 </pre>

14. State True or False (with reason):

- A. Every balanced binary tree is complete
- B. Every complete binary tree is balanced
- C. Every perfect binary tree is complete

Ans:

- A False
- B False
- C True

15. Different Traversal and their uses

Inorder: Use when you need sorted order or BST checking	- Getting sorted order from a BST - Validating BST **
Preorder: Use when you need to process root first	- Copying/Clone a tree ** - Creating expression trees - Serialization of trees
Postorder: Use when you need to process children before root (delete / evaluate)	- Deleting a tree ** - Evaluating expression trees - Bottom-up processing
Level Order: Use when you need level by level or shortest path (BFS)	- BFS - Finding height ** - Printing level-wise ** - Uses Queue **

16. Every complete binary tree is not always a full binary tree.

True.

**17. Is it possible to uniquely construct a binary tree using only:
Inorder traversal or Preorder traversal?**

Ans:

✗ No for both.

A tree is uniquely determined only when:

- Inorder + Preorder
- Inorder + Postorder

- Or it's BST

Basics

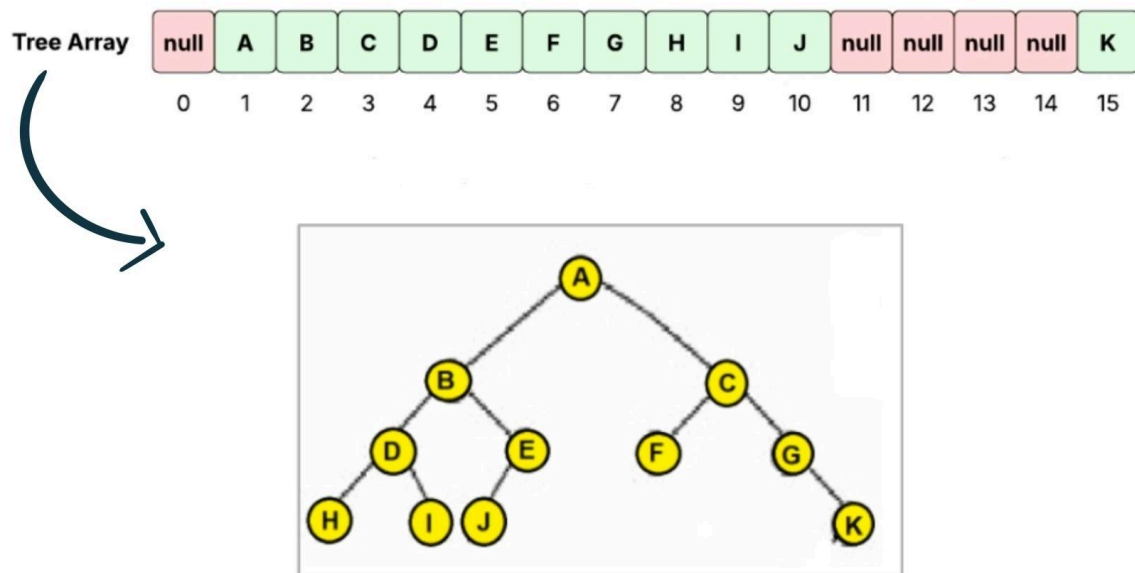
//Tree- Hierarchical Data Structure

Jargon = Terminology

1. BTreeNode Class

```
public class BTreeNode {  
    Object elem;  
    BTreeNode left;  
    BTreeNode right;  
  
    BTreeNode(Object elem) {  
        this.elem= elem;  
    }  
}
```

2. Convert Array to Tree



- If Height=h; Max length of array 2^{h+1}
- The root is at index = 1

- Node at i : Left Child at $2i$, Right Child at $2i+1$

```
public static BTNode createBT(Object[] arr, int index) {  
    if (index >= arr.length || arr[index] == null) {  
        return null;  
    }  
  
    BTNode node = new BTNode(arr[index]);  
  
    node.left = createBT(arr, 2 * index);  
    node.right = createBT(arr, 2 * index + 1);  
  
    return node;  
}
```

3. Traversal

PreOrder

```
public static void preorder(BTNode root) {  
    if (root == null) return;  
  
    System.out.print(root.elem + " ");  
    preorder(root.left);  
    preorder(root.right);  
}
```

InOrder

```

public static void inOrder( BTNode root ){
    if(root==null){
        return;
    }

    inOrder(root.left);
    System.out.print(root.elem + " ");
    inOrder(root.right);
}

```

PostOrder

```

public static void postorder(BTNode root) {
    if (root == null) return;

    postorder(root.left);
    postorder(root.right);
    System.out.print(root.elem + " ");
}

```

4a. Number of Nodes

```

public static int count(BTNode root) {
    if (root == null) return 0;

    return 1 + count(root.left) + count(root.right);
}

```

4b. Add all nodes

```

public static int sum(BTNode root) {
    if (root == null) return 0;

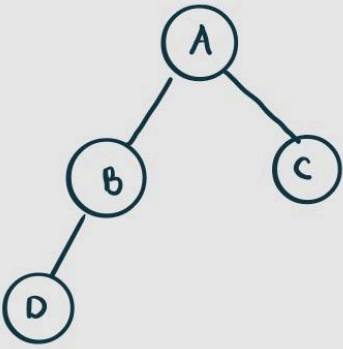
```

```

return (int) root.elem + sum(root.left) + sum(root.right);
}

```

5. Find Height (Top to farthest bottom)



```

public static int getH(BTnode node){
    ① if (node==null){
    ②     return -1;
    }
    ③ leftH = getH(node.left);
    ④ rightH = getH(node.right);
    ⑤ return Math.max(leftH, rightH)+1;
}

```

* Using height of B to understand

getH(A) ~~1~~ ~~2~~ ~~3~~ 4 5

↓

getH(B)

↓

getH(C)

1 2 3 4 5

0 -1 0+1=1

1 2 3 4 5

-1 -1 -1+1=0

```

public static int getHeight(BTNode node) {
    if (node == null) {

```

```

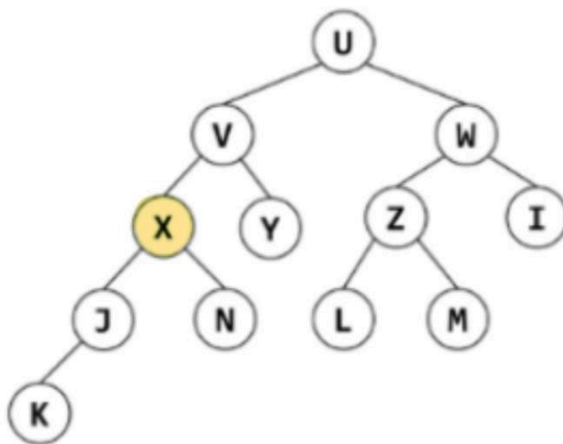
    return -1;
}

int leftHeight = getHeight(node.left);    //Last porjonto jabe
int rightHeight = getHeight(node.right);

return Math.max(leftHeight, rightHeight) + 1;
}

```

6. Find Level/ Depth



Find the depth/level of X

Ans:

} Base case
 } root is the target
 } left level
 } Right level

```

public static int levelOfNode(BTNode root, int target, int level) {
    if (root == null) return -1;    //Not found
    if ((int) root.elem == target) return level;

    int leftLevel = levelOfNode(root.left, target, level + 1);

```

```
int rightLevel = levelOfNode(root.right, target, level + 1);

if (leftLevel != -1) return leftLevel;
return rightLevel;
}
```

7. Find Occurrence of an Element

```
public static int occurrence(BTNode root, Object target){
    if (root == null) return 0;

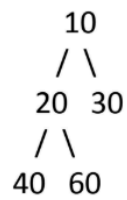
    int add = 0;
    if (root.elem.equals(target)) add = 1;

    return add + occurrence(root.left, target) + occurrence(root.right, target);
}
```

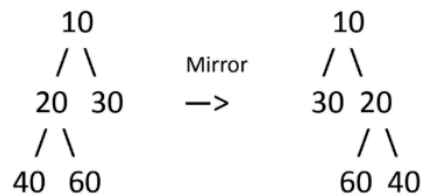
8. Convert a Tree to Its Mirror

Given a binary tree, convert it into its mirror.

Sample Input:



Sample Output:



Inorder Traversal of mirror: 30 10 60 20 40

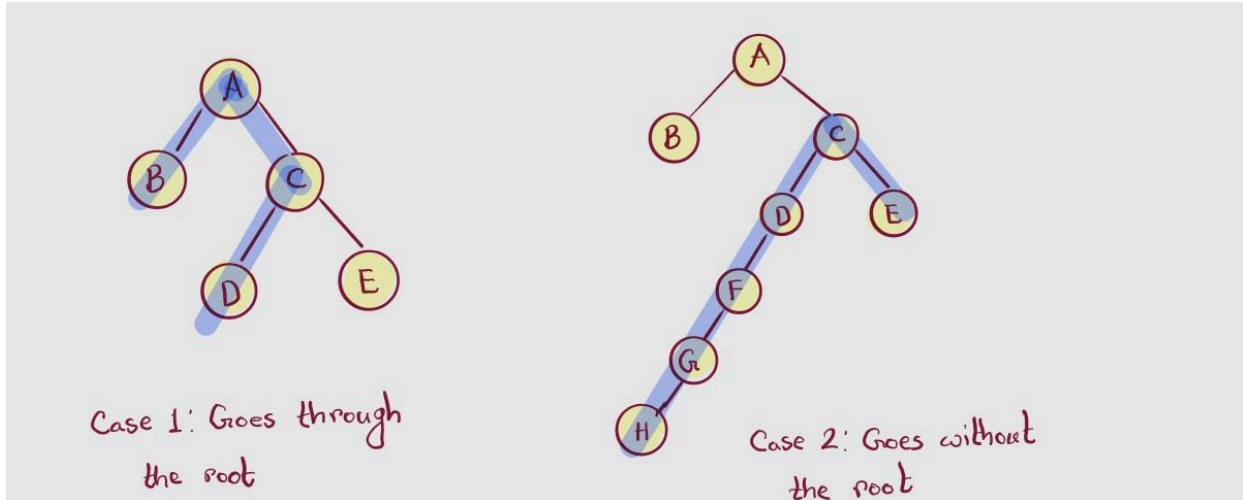
```
public static BTNode mirror(BTNode root){
    if (root == null) return null;

    BTNode left = mirror(root.left);
    BTNode right = mirror(root.right);

    root.left = right;
    root.right = left;

    return root;
}
```

9. Diameter



```
public static int diameter(BTNode root) {
    if (root == null) return 0;

    int throughRoot = height(root.left) + height(root.right) + 1; //Case 1
    int leftDiameter = diameter(root.left); //Case 2.1
    int rightDiameter = diameter(root.right); //Case 2.2

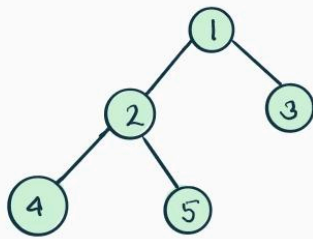
    return max(throughRoot, leftDiameter, rightDiameter);
}
```

```
private static int height(BTNode root) {
    if (root == null) return 0;

    return max(height(root.left), height(root.right)) + 1;
}
```

```
private static int max(int a, int b, int c) {
    return Math.max(a, Math.max(b, c));
}
```

10. Path Print



Target = 5

Output : 1 → 2 → 5

Iteration :

①

5
root.elem

① IF leftPath/ rightPath is ""
stop → Not found in that path

②

2 → 5
root.elem leftPath/
 rightPath

③

1 → 2 → 5
root.elem leftPath/rightPath

```

public static String pathToTarget(BTNode root, int target) {
    if (root == null) return ""; //Ending point 1

    if ((int) root.elem == target) { //Ending point 2
        return "" + root.elem;
    }

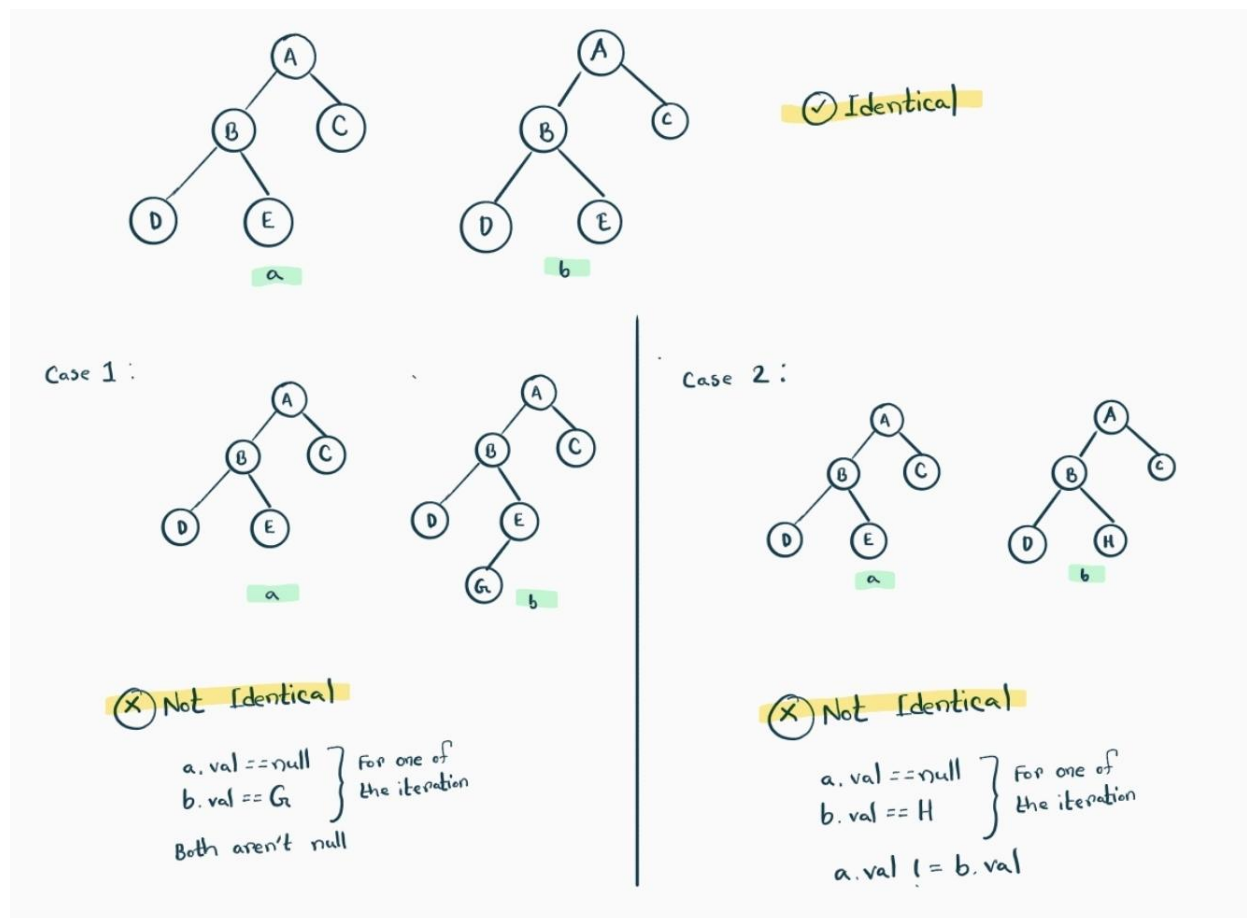
    String leftPath = pathToTarget(root.left, target);
    if (!leftPath.equals("")) {
        return root.elem + " -> " + leftPath;
    }

    String rightPath = pathToTarget(root.right, target);
    if (!rightPath.equals("")) {
        return root.elem + " -> " + rightPath;
    }

    return "";
}
  
```


Basics 2

1. Check if two trees are identical



```
public static boolean isIdentical(BTNode a, BTNode b){
```

```
//Case 1: Both should reach the end at the same time
```

```
if(a == null && b == null)
```

```
    return true;
```

```
if(a == null || b == null)
```

```
    return false;
```

```
//Case 2: Individual nodes should have same value
```

```
boolean left = isIdentical(a.left, b.left);
```

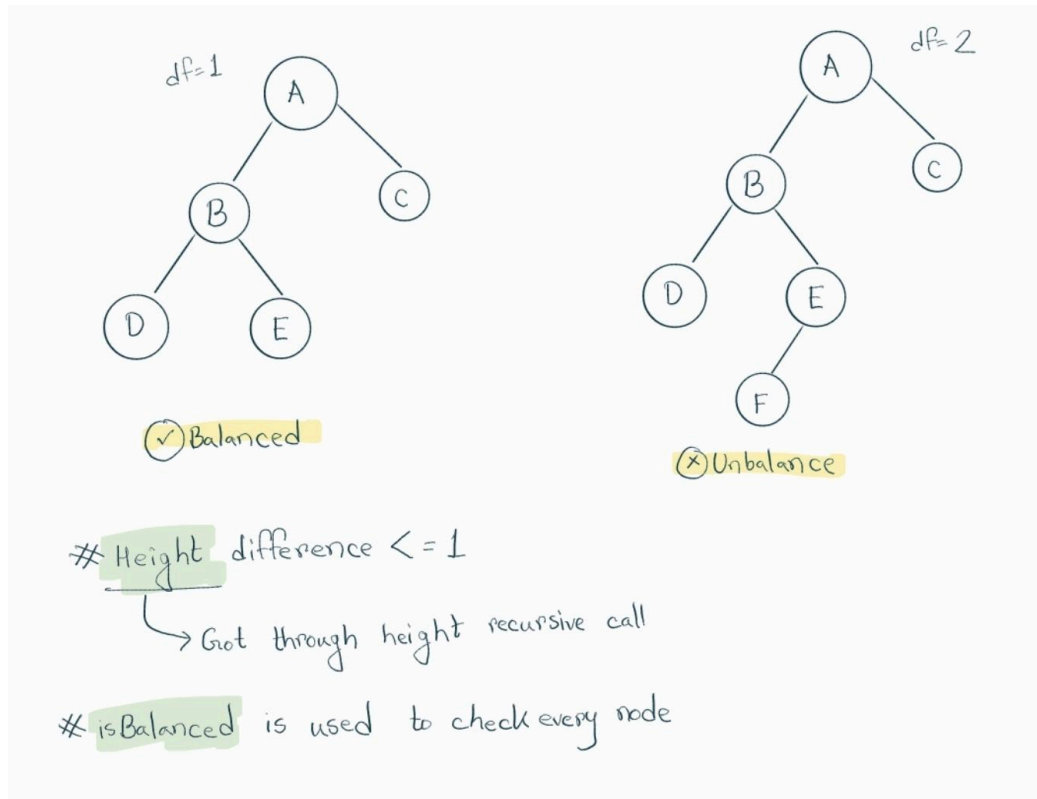
```
boolean right = isIdentical(a.right, b.right);
```

```
boolean valueMatch= (a.val == b.val); //postorder
```

```
return valueMatch && left && right;
```

}

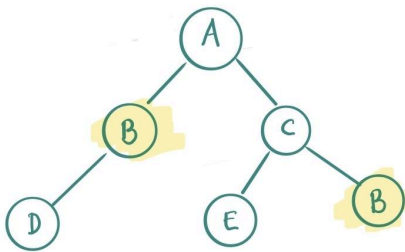
2. Check if a Binary Tree is Balanced



```
public static boolean isBalanced(BTNode root){  
    if(root == null) return true;  
  
    if(Math.abs(height(root.left) - height(root.right)) > 1)  
        return false;  
  
    return isBalanced(root.left) && isBalanced(root.right);  
}
```

```
private static int height(BTNode node){           //Uses concepts from Basics Question 5
    if(node == null) return 0;
    return 1 + Math.max(height(node.left), height(node.right));
}
```

3. Find Duplicate Elements



* B is a duplicate as it appears more than once

```
public static void findDuplicates(BTNode root){
    findDuplicatesHelper(root, root);
}
```

```
private static void findDuplicatesHelper(BTNode root, BTNode curr){
    if (curr == null) return;

    int c = count(root, curr.elem);
    if (c > 1) {
        System.out.print(curr.elem + " ");
    }

    findDuplicatesHelper(root, curr.left);
    findDuplicatesHelper(root, curr.right);
}
```


//Uses concepts from Basics Question 7

```
public static int count(BTNode root, Object value) {
    if (root == null) return 0;

    int c = 0;
    if (root.elem.equals(value)) {
        c = 1;
    }

    return c + count(root.left, value) + count(root.right, value);
}
```

4. Find the Unique Elements in a Binary Tree

(Opposite of Duplicate)

```
public static void findUnique(BTNode root){
    findUniqueHelper(root, root);
}

private static void findUniqueHelper(BTNode root, BTNode curr){
    if (curr == null) return;

    int c = count(root, curr.elem);
    if (c == 1) {
        System.out.print(curr.elem + " ");
    }

    findUniqueHelper(root, curr.left);
    findUniqueHelper(root, curr.right);
}
```

```

private static int count(BTNode node, Object value){
    if (node == null) return 0;

    int add = 0;
    if (node.elem.equals(value)) add = 1;

    return add + count(node.left, value) + count(node.right, value);
}

```

5. Check if Node Value = Sum of Children (Children Sum Property)

Sample Input	Sample Output	Explanation
<pre> 26 / \ 10 3 / \ \ 4 6 3 </pre>	True	For 26 Sum of left(20)+sum of right(6)=26 For 10 Sum of left(4)+Sum of right(6)=10 For 3 Sum of left(0)+Sum of right(3)=3

```

public static boolean childrenSum(BTNode root) {
    if (root == null || (root.left == null && root.right == null)) return true;

    int left = 0, right = 0;

    if (root.left != null){
        left = (int) root.left.elem;
    }
    if (root.right != null){
        right = (int) root.right.elem;
    }
}

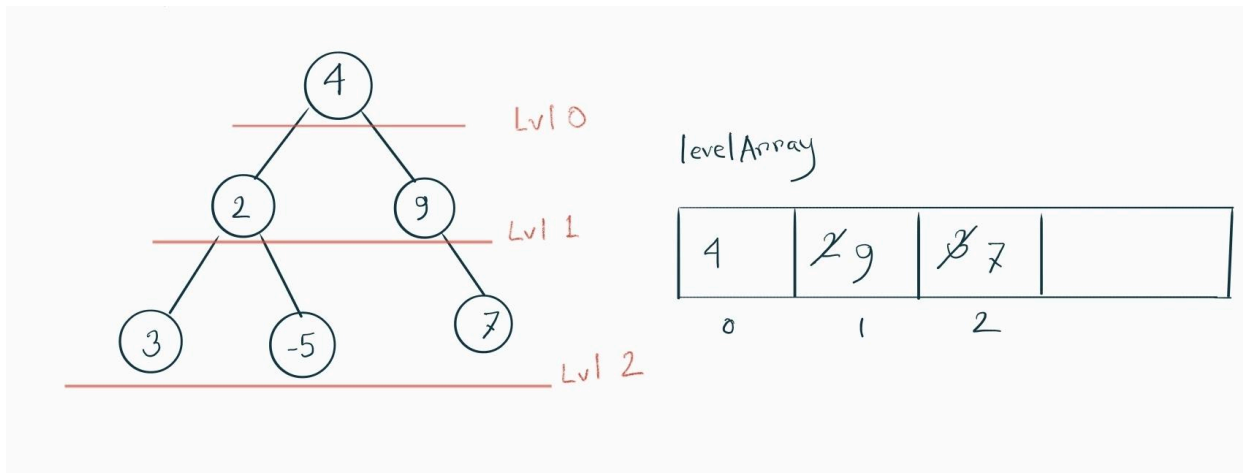
```

```

return ((int) root.elem == left + right) && childrenSum(root.left) &&
childrenSum(root.right);
}

```

6. Largest Value in Each Level



```

public static void largestEachLevel(BTNode root) {
    Integer[] levelArray = new Integer[5];    //5 is taken randomly

    fillLargest(root, levelArray, 0);

    int i = 0;
    while (i < levelArray.length && levelArray[i] != null) {
        System.out.print(levelArray[i] + " ");
        i++;
    }
}

```

```
private static void fillLargest(BTNode root, Integer[] levelArray, int lvl) {  
    if (root == null) return;  
  
    int value = (int) root.elem;  
    if (levelArray[lvl] == null || value > levelArray[lvl]) {  
        levelArray[lvl] = value;  
    }  
  
    fillLargest(root.left, levelArray, lvl + 1);  
    fillLargest(root.right, levelArray, lvl + 1);  
}
```

7. Level order traversal

Lab Task

Given a binary tree, find the smallest value in each level.

Sample Input: [For **Python** You can use a dictionary here]



Sample Output: 4 2 -5

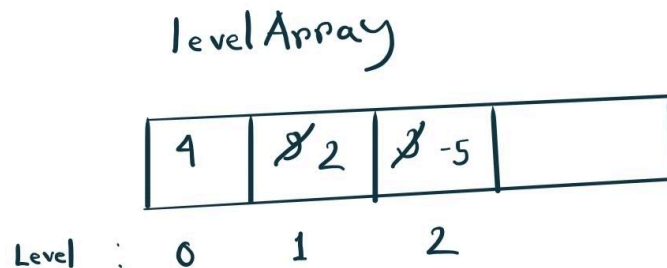
Explanation:

There are 3 levels in the tree

Level 0: {4}, min= 4

Level 1: {2, 9}, min= 2

Level 2: {7, 3, -5}, min = -5



(Similar to Basics 2 Question 6)

```
public static void smallestLevel(BTNode root, Integer[] levelArray, int lvl) {
    if (root == null) return;

    int value = (int) root.elem;

    if (levelArray[lvl] == null) {
        levelArray[lvl] = value;
    }

    else if (value < levelArray[lvl]) {
        levelArray[lvl] = value;
    }

    smallestLevel(root.left, levelArray, lvl + 1);
    smallestLevel(root.right, levelArray, lvl + 1);
}
```

Alice and Bob are having a competition on who is better at math between them. To decide who is better, Alice gave Bob a task of summation. Alice gave Bob a tree and asked him to sum the values each node consist. However, Alice added a condition with this task. Bob needs to divide the values of each node with the node's level and sum the modulus except for root node. For root node, he needs to sum the value of the node. Now, Bob finds it difficult to do it but he can not loose. Thus, he came to you for help. Your task is to write a method that takes the root of a tree and return the sum.

*** You can create as much helper function you want**

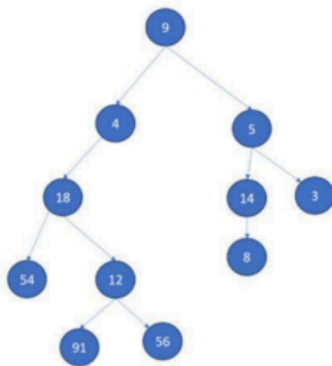
*** You can not modify the given codes nor the parameters.**

*** Hint: You know order traversal of the tree.**

Example:

Test 1

Input



Output

15

Explanation:

Level 0 → 9

Level 1 → $(4\%1) + (5\%1) = 0$

Level 2 → $(18\%2) + (14\%2) + (3\%2) = 1$

Level 3 → $(54\%3) + (12\%3) + (8\%3) = 2$

Level 4 → $(91 \% 4) + (56\%4) = 3$

Output → $9+0+1+2+3 = 15$

```

public static Integer sumTree(BTNode root) {
    return helper(root, 0);
}

```

```

private static int helper(BTNode node, int lvl) {
    if (node == null) return 0;

    int val = (int) node.elem;
    int sum;

    if (lvl == 0) {

```

```

        sum = val;    // root: take the value as is
    } else {
        sum = val % lvl;
    }

    sum += helper(node.left, lvl + 1);
    sum += helper(node.right, lvl + 1);

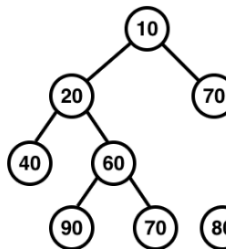
    return sum;
}

```

3. Print Kth Level of a Binary Tree [LAB TASK]:

Given the **root** of a binary tree and an integer K, print all the nodes that appear at the K-th level of the tree.

Sample Input:



K = 2

Sample Output:

Level 2 Nodes: **40, 60, 50**

```

public static void kthLevelPrint(BTNode root, int lvl) {
    printLevel(root, lvl);
}

```

// helper method

```

private static void printLevel(BTNode root, int lvl) {
    if (root == null) return;

```



```
if (lvl == 0) {                                // reached K-th level
    System.out.print(root.elem + " ");
    return;
}

printLevel(root.left, lvl - 1);
printLevel(root.right, lvl - 1);
}

}
```

Lab Assignment

4. Swap Children Nodes [ASSIGNMENT TASK]:

Write a **recursive** function **swap_child()** that takes the root of a binary tree, node's level and a number M as a parameter. The function will swap the left and right children of all the nodes at level M and above. Here, $0 < M < \text{height of the tree (root's height)}$. Consider, the Node class for Binary Tree already defined with elem, left and right variables. **YOU CANNOT USE LIST OR DICTIONARY, any built-in function, global variables.**

Python Notation:

```
def swap_child(root, level, M):  
    # To do
```

Function Call :

swap_child(root, 0, 2). Here root refers to the tree below.

Input Tree	Resulting Tree	Explanation
		Here $M = 2$ and all the nodes from level 2 and above are swapped left with right. Here above means the level that situated at a higher position of the tree

```
public static void swapChild( BTreeNode root, int lvl, int M ){  
    helper(root, lvl, M);  
}
```

```
private static BTreeNode helper(BTreeNode root, int lvl, int M){  
    if (root==null){  
        return null;  
    }  
}
```

```
    BTreeNode left= helper(root.left, lvl+1, M-1);  
    BTreeNode right= helper(root.right, lvl+1, M-1);
```

```
    if(M >0){
```

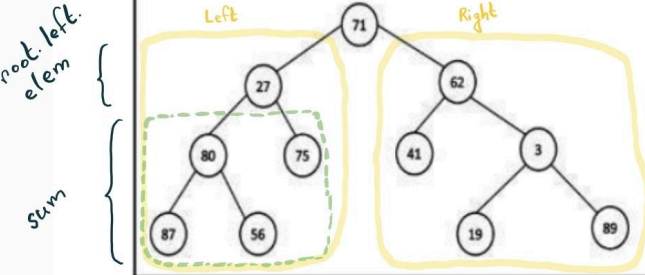
```

        root.left=right;
        root.right=left;
    }

    else{
        root.left=left;
        root.right=right;
    }

    return root;
}

```

	Sample Input	Sample Output	Explanation
		111	Summation of left subtree - summation of right subtree $= (27+75+80+87+56)$ $- (62+41+3+19+89)$ $= 111$

```

public static Integer subtractSummation( BTreeNode root ){
    Integer left= (Integer) root.left.elem + sum(root.left);
    Integer right= (Integer) root.right.elem + sum(root.right);
    return left-right;
}

private static Integer sum(BTreeNode root){
    if(root==null){
        return 0;
    }
}

```

```

}
Integer currLeft=0,currRight=0;

if(root.left!=null){
    currLeft= (Integer)root.left.elem;
}

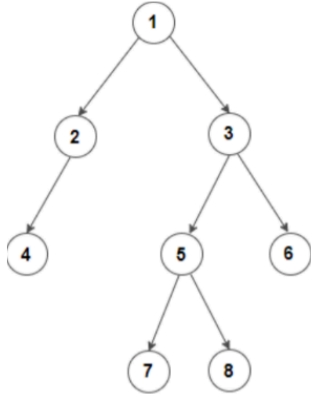
if(root.right!=null){
    currRight= (Integer)root.right.elem;
}

return currLeft + currRight + sum(root.left)+ sum(root.right);
}

```

6. Difference of Level Sum [ASSIGNMENT TASK]:

Given a Binary Tree, Write a function that finds the difference between sum of all nodes present at odd and even levels in a binary tree, i.e. sum of all odd level nodes - sum of all even level nodes.

Sample Input:	Sample Output	Explanation
 <pre> graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 3 --> 5((5)) 3 --> 6((6)) 5 --> 7((7)) 5 --> 8((8)) </pre>	4	$-1+2+3-4-5-6+7+8 = 4$

```

public static Integer levelSum( BTreeNode root ){
    Integer sum= helper(root, 0);
}

```

```

        return sum;
    }

    private static Integer helper(BTNode root, int level){
        if(root==null){
            return 0;
        }

        Integer value=0;
        if(level%2==0){
            value= ((-1)*(Integer) root.elem);    //return hobe na ei line e
        }

        if(level%2!=0){
            value= (Integer) root.elem;
        }

        return value + helper(root.left, level+1) + helper(root.right, level+1);
    }

```

Lab Test

Create a function called `strangeFunc` which takes a Binary tree root and an integer `k` as input: `strangeFunc (Node root, int k)`. Then recursively, replace each node by the absolute difference of the child nodes if the node is divisible by `k`, else double them. If a node has a missing child, do not apply this algorithm to it or its subtrees.

(Assume node class is already implemented)

Absolute Function

Python: `abs(num)`

Java: `Math.abs(num)`

Sample Input	Sample Output
<code>strangeFunc (root, 3)</code> <pre> graph TD 2 --> 3 2 --> 5 3 --> 6 3 --> 7 5 --> 8 5 --> 9 6 --> 10 8 --> 11 8 --> 12 9 --> 14 </pre>	Here 2 is not divisible by 3, hence it is doubled. 3 is divisible by 3, hence it is replaced by the absolute difference of its child. 6 and 9 both have missing children, so the algorithm is not applied to those subtrees.. <pre> graph TD 4 --> 1 4 --> 10 1 --> 6 1 --> 7 10 --> 16 10 --> 9 6 --> 10 16 --> 11 16 --> 12 9 --> 14 </pre>

```

public static void strangeFunc(Node root, int k) {
    if (root == null) return;

    if (root.left == null || root.right == null) return;

    if (root.elem % k != 0) {
        root.elem = root.elem * 2;
    }
    else {
        int diff = Math.abs(root.left.elem - root.right.elem);
        root.elem = diff;
    }

    strangeFunc(root.left, k);
    strangeFunc(root.right, k);
}

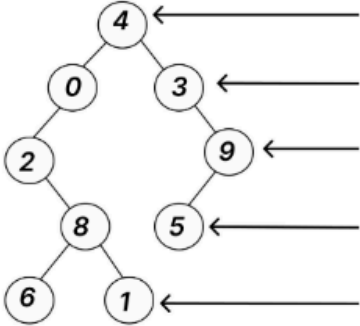
```


You're developing an AI agent that observes a **Binary Tree battlefield** from the right side. The agent can only "see" nodes that are visible from the right side of the tree.

Your task is to write a *recursive function/method* **RightView(root, level, max_Level)** that prints a list of node values the agent can see, ordered from top to bottom.

- Here, root: The current node, level: The current level of the node in the tree, max_level: A variable to store the deepest level encountered so far.
- Initially, root = the root of the binary tree, level = 0, and max_level = -1.
- You may use helper functions if needed.
- Think of the tree levels as "rows" and always prioritize seeing the rightmost node first at each row.

YOU CANNOT USE LIST OR DICTIONARY. You cannot use any built-in function.

Sample Input	Sample Output
	4 3 9 5 1 Explanation: As the arrows indicate, if you see the graph from the right side, then the first nodes that you encounter at each level are the nodes you need to print from top to bottom.

```
public static void RightView(BTNode root, int level, int max_level) {
    if (root == null) return;

    int[] arr = new int[10];
    rightmost(root, level, arr);

    for (int i = 0; i < arr.length; i++) {
        if (arr[i] != 0) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

```
}
}
```

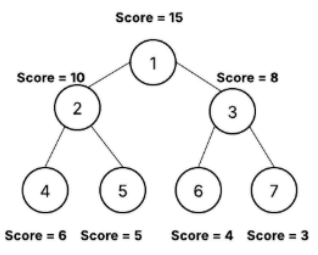
```
private static void rightmost(BTNode root, int level, int[] a) {
    if (root == null) return;

    a[level] = root.elem;
    rightmost(root.left, level + 1, a);
    rightmost(root.right, level + 1, a);
}
```

The BRAC University Computer Club (BUCC) is organized as a binary tree, where each member has a unique ID and a contribution score representing their effort for the club. A member may have up to two direct juniors: the first junior is the left child, and the second junior is the right child.

Given the ID of a particular member, compute the total contribution score of that member and all of their direct and indirect juniors in the binary tree. If the given ID does not exist in the binary tree, return 0. Write a function `totalContribution()` that takes the root of the binary tree and the target ID as its parameters.

Note: You may use extra parameters or helper functions as much as you want. You are required to use recursion. Consider the `BTNode` class is already implemented with the instance variables `left (BTNode)`, `right (BTNode)`, `score (int)`, `ID (int)`.

Sample Tree	Sample Function Call	Sample Output
	(Python) <code>print(total_contribution(root, 1))</code>	51
	(Java) <code>System.out.println(totalContribution(root, 1));</code>	Explanation: (15+10+8+6+5+4+3)=51
	(Python) <code>print(total_contribution(root, 2))</code>	21
	(Java) <code>System.out.println(totalContribution(root, 2));</code>	Explanation: (10+6+5)=21

```
public static int totalContribution(BTNode root, int target) {
    if (root == null) return 0;

    if (root.ID == target) {
```

```
        return sumSubtree(root);
    }

    int leftSum = totalContribution(root.left, target);
    int rightSum = totalContribution(root.right, target);

    if (leftSum != 0) return leftSum;
    if (rightSum != 0) return rightSum;

    return 0;
}
```

```
private static int sumSubtree(BTNode root) {
    if (root == null) return 0;

    return root.score
        + sumSubtree(root.left)
        + sumSubtree(root.right);
}
```

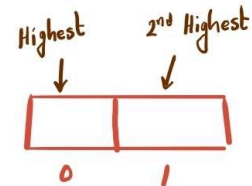
RFTS Practice

Complete the function below, which will receive the root of a binary tree as a parameter and return the second-highest value from the tree. Assume there will always be a second-highest value in the tree. **The node class has an elem (Integer type), left (Node type), and right (Node type) variable.**

Restrictions:

- You cannot use any global variables and cannot create any other data structures (e.g. Array length more than 2, linked list, etc).
- You cannot change the number of parameters of the second_max function.
- You can design and use helper functions. You can use an array of length 2 if you want to.

Input tree	Returned Value	Explanation
<pre> 30 / \ 50 10 / 40 </pre>	40	In the given binary tree, the maximum value is 50 and the second-highest value is 40.



(A bit different)

```

public static Integer second_max(BTNode root) {
    int[] arr = { root.elem, root.elem };

    helper(root, arr);    //Just update the array values, no need to return + store
                           anything here

    return arr[1];
}

```

```

private static void helper(BTNode node, int[] arr) {
    if (node == null) return;

    if ((int) node.elem > arr[0]) {
        arr[1] = arr[0];
        arr[0] = (int) node.elem;
    }

    else if ((int) node.elem > arr[1] &&
        (int) node.elem < arr[0]) {
        arr[1] = (int) node.elem;
    }
}

```

```

    helper(node.left, arr);
    helper(node.right, arr);
}

```

You are given a binary tree where each node stores a character. You need to generate a key based on the leaf nodes positioned at the even levels **from right to left**. Your task is to implement a **function/method** named **build_key** that takes the root of the binary tree and constructs the key string based on the criteria above and finally returns it. If there are no leaf nodes on even levels, then the function/method should return an empty string.

Consider the Node class for Binary Tree already defined with elem (Type: String), left (Type: Node), and right (Type: Node) variables. **You can use helper functions or extra parameters in the given function if required. You can not use any other data structures than the given binary tree.**

Input tree	Returned String	Explanation
<pre> graph TD U((U)) --> V((V)) U --> W((W)) V --> X((X)) V --> Y((Y)) X --> J((J)) X --> N((N)) J --> K((K)) W --> Z((Z)) W --> I((I)) Z --> L((L)) Z --> M((M)) </pre>	IYK	All the leaf nodes from right to left are: I, M, L, Y, N, K However, only I, Y, K are on even levels. Hence the output string is "IYK"

```

public static String build_key(BTNode root) {
    return helper(root, 0);
}

```

```

private static String helper(BTNode node, int level) {
    if (node == null) return "";

    if (node.left == null && node.right == null) {

```

```
    if (level % 2 == 0) {  
        return (String) node.elem;  
    }  
    else{  
        return "";  
    }  
}  
  
String rightPart = helper(node.right, level + 1);  
String leftPart = helper(node.left, level + 1);  
  
return rightPart + leftPart;  
}
```

In a maze game, a player starts at the entry point of the maze, which is the root of a binary tree. Each node contains either a 0 or a 1. Each path from the root down to a leaf represents one possible way out through the maze. The player is given a sequence of 0s and 1s. They must check whether following that sequence will allow them to escape the maze by reaching a leaf node.

Write a function **findPath()** to help the player. This function will take the **root** of the binary tree, and the **sequence in an (int) array** to follow. The function will **return a string** "ESCAPED" if the sequence matches a complete path from the root to a leaf, "TRAPPED" if the sequence matches part of a path but ends at a non-leaf node, "LOST" if the sequence cannot be matched to any path in the tree.

Assume the Node class of the binary tree is already defined with the properties: (int) elem, (Node) left, (Node) right.

- You can take only one extra parameter or a helper function.
- You must solve this using recursion only.
- No extra data structures (array/linked list) or static/global variables are allowed.

Tree	Input	Returned String	Explanation
	seq = {1,0,1,1} findPath(root, seq)	ESCAPED	Matches path 1 → 0 → 1 → 1 Completely from root to leaf
	seq = {1,1,0} findPath(root, seq)	TRAPPED	Matches part of the path but ends at a non-leaf
	seq = {1,0,0,1} findPath(root, seq)	LOST	No such path in this tree

Python Notation	Java Notation
<pre>def findPath(root, seq): #To do</pre>	<pre>public static String findPath(Node root, int[] seq){ //To do }</pre>

```
public static String findPath(Node root, int[] seq) {
    return helper(root, seq, 0);
}
```

```
private static String helper(Node node, int[] seq, int index) {
```

```
    // Case 1: No node
```

```
    if (node == null) return "LOST";
```

```
    // Case 2: Value mismatch
```

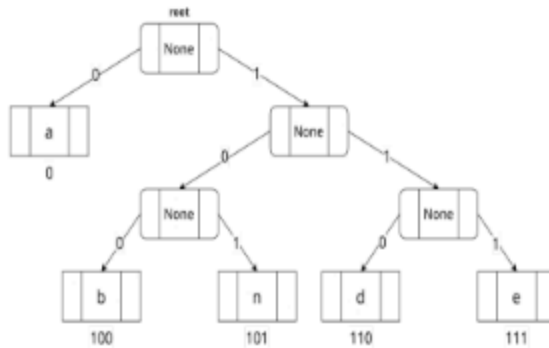
```
    if (node.elem != seq[index]) return "LOST";
```


// Case 3: Sequence finished

```
if (index == seq.length - 1) {  
    if (node.left == null && node.right == null) {  
        return "ESCAPED";  
    } else {  
        return "TRAPPED";  
    }  
}  
  
String leftResult = helper(node.left, seq, index + 1);  
if (leftResult.equals("ESCAPED")) return "ESCAPED";  
if (leftResult.equals("TRAPPED")) return "TRAPPED";  
  
return helper(node.right, seq, index + 1);  
}
```

Question – 6: Bonus [5 Points]

The binary tree, called the Huffman coding tree, has all the possible valid characters as the leaves and all the internal nodes contain None/null. Each transition to a left child represents a string 0-value bit and each transition to a right child from the parent represents a string 1-value bit. The bit representation of a character is the sequence of 0s and 1s you get through the path from the root to the leaf holding the characters.



Here, the representation of **a** would be 0. Since we're moving only to the left 1 time. Similarly, **n** would be 101 since we're moving right, left, and right, and **d** would be 110 since we're moving right, right, left.

Each node of the Huffman encoding tree has four properties, a parent property to hold a reference of the parent of the current node, a left and a right to refer to the two children, and finally, a value property which is a character for the leaves and None/Null for the internal nodes.

Your task is to implement the following recursive function that takes the root of a Huffman encoding tree and a string text as parameters and returns a string representing the compressed bit encoding of the input string.

[The parent property of the node is absolutely not necessary to solve the task. But you can optimize the solution of this by using the node's parent property]

Note: You can create other recursive helper functions and call them inside the given input function. You can also use loops in your recursive functions. **There is no restriction on using other data structures in this problem.**

Sample Function Call	Sample Returned String	Explanation
<code>getEncoding(root, "and")</code>	<code>"0101110"</code>	Here, the bit representation of strings of "a", "n", "d" are "0", "101", "110" respectively. So, they're concatenated and shown in the output.

Python Notation	Java Notation
<pre>def getEncoding(root, st): #To do</pre>	<pre>static String getEncoding(Node root, String st){ //To do }</pre>